CHAPTER

A Brief Introduction to OpenCL™

11

CHAPTER CONTENTS

11.1 Background 205
11.2 Data Parallelism Model 207
11.3 Device Architecture 209
11.4 Kernel Functions 211
11.5 Device Management and Kernel Launch 212
11.6 Electrostatic Potential Map in OpenCL 214
11.7 Summary 219
11.8 Exercises 220
References and Further Reading 220

INTRODUCTION

Now that you have learned high-performance parallel programming using CUDA™, we would like to introduce you to two new ways to exploit the parallel computing capabilities of graphics processing units (GPUs) and central processing units (CPUs). During the writing of this book, two new application programming interfaces (APIs) were introduced: OpenCL™ and DirectCompute™. In this chapter, we will give a brief overview of OpenCL to CUDA programmers. The fundamental programming model of OpenCL is so similar to CUDA that there is a one-to-one correspondence for most features. With your understanding of CUDA, you will be able to start writing OpenCL programs with the material presented in this chapter. In our opinion, the best way to learn OpenCL is actually to learn CUDA and then map the OpenCL features to their CUDA equivalent. We will then offer some pointers to introductory information about Microsoft's DirectCompute.

11.1 BACKGROUND

OpenCL is a standardized, cross-platform, parallel-computing API based on the C language. It is designed to enable the development of portable parallel applications for systems with heterogeneous computing devices.

The development of OpenCL was motivated by the need for a standardized high-performance application development platform for the fast-growing variety of parallel-computing platforms. In particular, it addresses significant limitations of the previous programming models for heterogeneous parallel-computing systems.

The CPU-based parallel programming models have typically been based on standards such as OpenMP™ but usually do not encompass the use of special memory types or SIMD execution by high-performance programmers. Joint CPU/GPU heterogeneous parallel programming models such as CUDA address complex memory hierarchies and SIMD execution but are typically platform, vendor, or hardware specific. These limitations make it difficult for an application developer to access the computing power of CPUs, GPUs, and other types of processing units from a single multiplatform source-code base.

The development of OpenCL was initiated by Apple® and developed by the Khronos Group, the same group that manages the OpenGL® standard. On one hand, it draws heavily on CUDA in the areas of supporting a single code base for heterogeneous parallel computing, data parallelism, and complex memory hierarchies. This is the reason why a CUDA programmer will find these aspects of OpenCL familiar once we connect the terminologies. The reader will especially appreciate the similarities between OpenCL and the low-level CUDA driver model.

On the other hand, OpenCL has a more complex platform and device management model that reflects its support for multiplatform and multivendor portability. OpenCL implementations already exist on AMD® ATI™ and NVIDIA® GPUs as well as x86 CPUs. In principle, one can envision OpenCL implementations on other types of devices such as digital signal processors (DSPs) and field-programmable gate arrays (FPGAs). Whereas the OpenCL standard is designed to support code portability across devices produced by different vendors, such portability does not come free. OpenCL programs must be prepared to deal with much greater hardware diversity and thus will exhibit more complexity. Also, many OpenCL features are optional and may not be supported on all devices, so a portable OpenCL code must avoid using these optional features. Some of these optional features, though, allow applications to achieve significantly more performance in devices that support them. As a result, a portable OpenCL code may not be able to achieve its performance potential on any of the devices; therefore, one should expect that a portable application that achieves high performance on multiple devices will employ sophisticated runtime tests and choose among multiple code paths according to the capabilities of the actual device used.

The objective of this chapter is not to provide full details on all programming features of OpenCL; rather, the objective is to give a CUDA programmer solid conceptual understanding of the OpenCL programming model features. With this foundation, the reader can immediately begin to program in OpenCL and consult the OpenCL specification [Khronos 2008] and programming guides [NVIDIA 2010, AMD 2009] on a needs basis.

11.2 DATA PARALLELISM MODEL

OpenCL employs a data parallelism model that has direct correspondence with the CUDA data parallelism model. An OpenCL program consists of two parts: kernels that execute on one or more OpenCL devices and a host program that manages the execution of kernels. Figure 11.1 summarizes the mapping of OpenCL data parallelism concepts to their CUDA equivalents. Like CUDA, the way to submit work for parallel execution in OpenCL is for the host program to launch kernel functions. We will discuss the additional kernel generation, device selection, and management work that an OpenCL host program must do as compared to its CUDA counterpart in Section 11.4.

When a kernel function is launched, its code is run by *work items*, which correspond to CUDA threads. An index space defines the work items and how data are mapped to the work items; that is, OpenCL work items are identified by global dimension index ranges (NDRanges). Work items form *work groups*, which correspond to CUDA thread blocks. Work items in the same work group can synchronize with each other using barriers that are equivalent to `syncthreads()` in CUDA. Work items in different work groups cannot synchronize with each other except by terminating the kernel function and launching a new one. As we discussed in Chapter 4, this limited scope of barrier synchronization enables transparent scaling.

OpenCL parallelism concept	CUDA equivalent
Kernel	Kernel
Host program	Host program
NDRange (index space)	Grid
Work item	Thread
Work group	Block

FIGURE 11.1

Mapping between OpenCL and CUDA data parallelism model concepts.

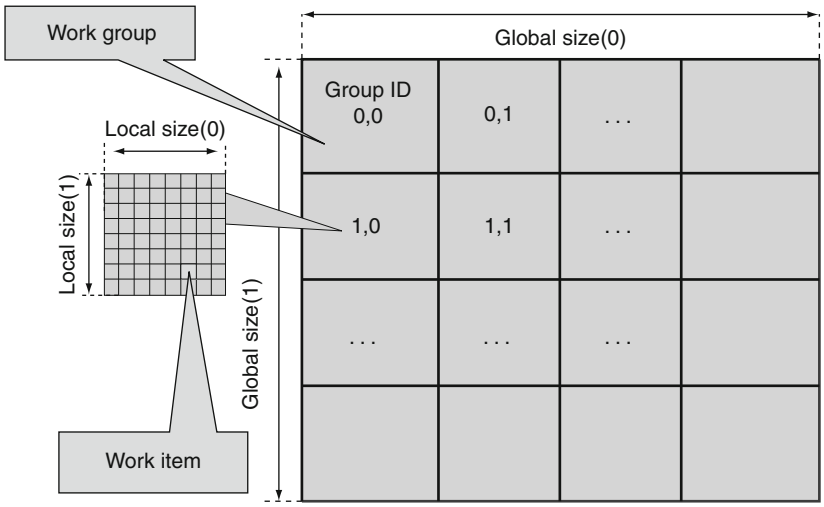

FIGURE 11.2

Overview of the OpenCL parallel execution model.

Figure 11.2 illustrates the OpenCL data parallelism model. The reader should compare Figure 11.2 with Figure 9.8 for similarities. The NDRange (CUDA grid) contains all work items (CUDA threads). For this example, we assume that the kernel is launched with a two-dimensional (2D) NDRange.

All work items have their own unique global index values. There is a minor difference between OpenCL and CUDA in the way they manage these index values. In CUDA, each thread has a `blockIdx` value and a `threadIdx` value. The two values are combined to form a global thread ID value for the thread. For example, if a CUDA grid and its blocks are organized as 2D arrays, the kernel code can form a unique global thread index value in the *x*-dimension as `blockIdx.x × blockDim.x + threadIdx.x`. These `blockIdx` and `threadIdx` values are accessible in a CUDA kernel as predefined variables.

In an OpenCL kernel, a thread can get its unique global index values by calling an API function, `get_global_id()`, with a parameter that identifies the dimension. See the `get_global_id(0)` entry in Figure 11.3. The calls `get_global_id(0)` and `get_global_id(1)` return the global thread index values in the *x*-dimension and the *y*-dimension, respectively. The global index value in the *x*-dimension is equivalent to the `blockIdx.x × blockDim.x + threadIdx.x` in CUDA. See Figure 11.3 for the `get_local_id(0)` function,

OpenCL API call	Explanation	CUDA equivalent
get_global_id(0);	Global index of the work item in the *x*-dimension	blockIdx.x×blockDim.x+threadIdx.x
get_local_id(0)	Local index of the work item within the work group in the *x*-dimension	threadIdx.x
get_global_size(0);	Size of NDRange in the *x*-dimension	gridDim.x ×blockDim.x
get_local_size(0);	Size of each work group in the *x*-dimension	blockDim.x

FIGURE 11.3

Mapping of OpenCL dimensions and indices to CUDA dimensions and indices.

which is equivalent to `threadIdx.x`. We did not show the higher dimension parameter values in Figure 11.3; they are 1 for the *y*-dimension and 2 for the *z*-dimension.

An OpenCL kernel can also call the API function `get_global_size()` with a parameter that identifies the dimensional sizes of its NDRanges. The calls `get_global_size(0)` and `get_global_size(1)` return the total number of work items in the *x*- and *y*-dimensions of the NDRanges. Note that this is slightly different from the CUDA `gridDim` values, which are in terms of blocks. The CUDA equivalent for the `get_global_size(0)` return value would be `gridDim.x×blockDim.x`.

11.3 DEVICE ARCHITECTURE

Like CUDA, OpenCL models a heterogeneous parallel computing system as a host and one or more *OpenCL devices*. The host is a traditional CPU that executes the host program. Figure 11.4 shows the conceptual architecture of an OpenCL device. Each device consists of one or more *compute units* (CUs) that correspond to CUDA streaming multiprocessors (SMs); however, a CU can also correspond to CPU cores or other types of execution units in compute accelerators such as DSPs and FPGAs.

Each CU, in turn, consists of one or more *processing elements* (PEs), which correspond to the streaming processors (SPs) in CUDA. Computation on a device ultimately happens in individual PEs. The reader should compare Figure 11.4 with Figure 3.7 for similarities between the

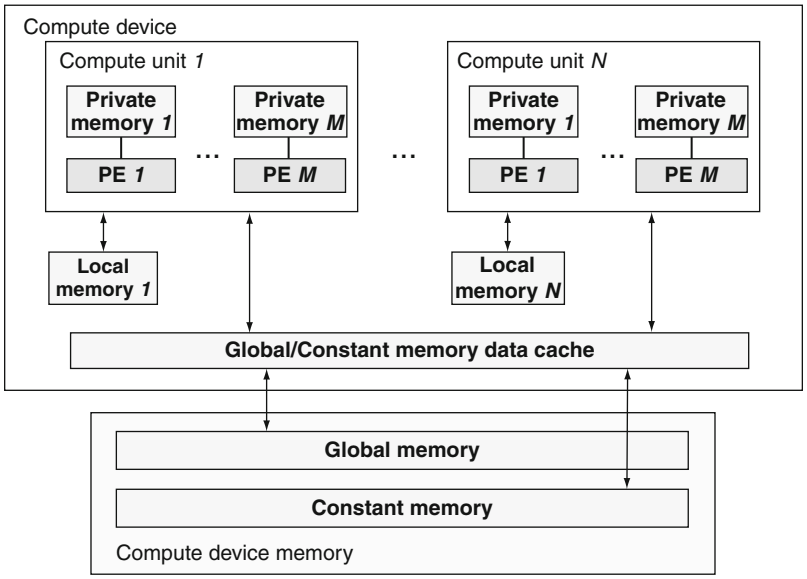

FIGURE 11.4

Conceptual OpenCL device architecture; the host is not shown.

OpenCL device architecture model and the CUDA device architecture model.

Like CUDA, OpenCL also exposes a hierarchy of memory types that can be used by programmers. Figure 11.4 illustrates these memory types: global, constant, local, and private. Figure 11.5 summarizes the supported use of OpenCL memory types and the mapping of these memory types to CUDA memory types. The OpenCL global memory corresponds to the

OpenCL Memory Types	CUDA Equivalent
global memory	global memory
constant memory	constant memory
local memory	shared memory
private memory	Local memory

FIGURE 11.5

Mapping of OpenCL memory types to CUDA memory types.

CUDA global memory. Like CUDA, the global memory can be dynamically allocated by the host program and supports read/write access by both host and devices.

Unlike CUDA, the constant memory can be dynamically allocated by the host. The constant memory supports read/write access by the host and read-only access by devices. To support multiple platforms, the size of constant memory is not limited to 64 kB in OpenCL. Instead, a device query returns the constant memory size supported by the device.

The mapping of OpenCL local memory and private memory to CUDA memory types is more interesting. The OpenCL local memory actually corresponds to CUDA shared memory. The OpenCL local memory can be dynamically allocated by the host and statically allocated in the device code. Like the CUDA shared memory, the OpenCL local memory cannot be accessed by the host and support shared read/write access by all work items in a work group. The private memory of OpenCL corresponds to the CUDA local memory.

11.4 KERNEL FUNCTIONS

OpenCL kernels have identical basic structure as CUDA kernels. All OpenCL kernel function declarations start with a `__kernel` keyword, which is equivalent to the `__global` keyword in CUDA. Figure 11.6 shows a simple OpenCL kernel function that performs vector addition. The function takes three arguments: pointers to the two input arrays and one pointer to the output array. The `__global` declarations in the function header indicate that the input and output arrays all reside in the global memory. Note that this keyword has the same meaning in OpenCL as in CUDA. The body of the kernel function is instantiated once for each work item. In Figure 11.6, each work item calls the `get_global_id(0)` function to receive its unique global index. This index value is then used by the work item to select the array elements to work on.

```
__kernel void vadd(__global const float *a,
     __global const float *b, __global float *result) {

      int id = get_global_id(0);
      result[id] = a[id] + b[id];
}
```

FIGURE 11.6

A simple OpenCL kernel example.

11.5 DEVICE MANAGEMENT AND KERNEL LAUNCH

OpenCL defines a much more complex model of device management than CUDA. The extra complexity stems from the OpenCL support for multiple hardware platforms. In OpenCL, devices are managed through *contexts*. Figure 11.7 illustrates the main concepts of device management in OpenCL. In order to manage one or more devices in the system, the OpenCL programmer first creates a context that contains these devices. This can be done by calling either `clCreateContext()` or `clCreateContextFromType()` in the OpenCL API. An application typically needs to use the `clGetDeviceIDs()` API function to determine the number and types of devices that exist in a system and to pass the information on to the `CreateContext` functions. The reader should read the OpenCL programming guide regarding the details of the parameters to be used for these functions.

To submit work for execution by a device, the host program must first create a command queue for the device. This can be done by calling the `clCreateCommandQueue()` function in the OpenCL API. Once a command queue is created for a device, the host code can perform a sequence of API function calls to insert a kernel along with its execution configuration parameters into the command queue. When the device is available for executing the next kernel, it removes the kernel at the head of the queue for execution.

Figure 11.8 shows a simple host programs that creates a context for a device and submits a kernel for execution by the device. Line 2 shows a call to create a context that includes all OpenCL available devices in

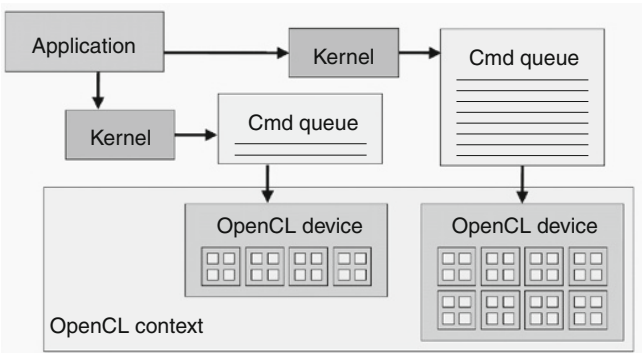

FIGURE 11.7

OpenCL context required to manage devices.

```
...
1. cl_int clerr = CL_SUCCESS;

2. cl_context clctx=clCreateContextFromType(0, CL_DEVICE_TYPE_ALL,
          NULL, NULL, &clerr);

3. size_t parmsz;
4. clerr= clGetContextInfo(clctx, CL_CONTEXT_DEVICES, 0, NULL, &parmsz);

5. cl_device_id* cldevs= (cl_device_id *) malloc(parmsz);
6. clerr= clGetContextInfo(clctx, CL_CONTEXT_DEVICES, parmsz,cldevs, NULL);

7. cl_command_queue clcmdq=clCreateCommandQueue(clctx,cldevs[0], 0, &clerr);
```

FIGURE 11.8

Creating an OpenCL context and command queue.

the system. Line 4 calls the `clGetContextInfo()` function to inquire about the number of devices in the context. Because Line 2 asks that all OpenCL available devices be included in the context, the application does not know the number of devices actually included in the context after the context is created. The second argument of the call in Line 4 specifies that the information being requested is the list of all devices included in the context; however, the fourth argument, which is a pointer to a memory buffer where the list should be deposited, is a `NULL` pointer. This means that the call does not want the list itself. The reason is that the application does not know the number of devices in the context and does not know the size of the memory buffer required to hold the list.

Line 4 provides a pointer to the variable `parmsz`, where the size of a memory buffer required to accommodate the device list is to be deposited; therefore, after the call in Line 4, the `parmsz` variable holds the size of the buffer needed to accommodate the list of devices in the context. The application now knows the amount of memory buffer needed to hold the list of devices in the context. It allocates the memory buffer using `parmsz` and assigns the address of the buffer to pointer variable `cldevs` at Line 5.

Line 6 calls `clGetContextInfo()` again with the pointer to the memory buffer in the fourth argument and the size of the buffer in the third argument. Because this is based on the information from the call at Line 4, the buffer is guaranteed to be the right size for the list of devices to be returned. The `clGetContextInfo` function now fills the device list information into the memory buffer pointed to by `cldevs`.

Line 7 creates a command queue for the first OpenCL device in the list. This is done by treating `cldevs` as an array whose elements are descriptors

of OpenCL devices in the system. Line 7 passes `cldevs[0]` as the second argument into the `clCreateCommandQueue(0)` function; therefore, the call generates a command queue for the first device in the list returned by the `clGetContextInfo()` function.

The reader may wonder why we did not need to see this complex sequence of API calls in our CUDA host programs. The reason is that we have been using the CUDA runtime API that hides all of this complexity for the common case where there is only one CUDA device in the system. The kernel launch in CUDA handles all of the complexities on behalf of the host code. A developer who wants to have direct access to all CUDA devices in the system would need to use the CUDA driver API, where similar API calling sequences would be used. To date, OpenCL has not defined a higher level API that is equivalent to the CUDA runtime API. Until such a higher level interface is available, OpenCL will remain much more tedious to use than the CUDA runtime API.

11.6 ELECTROSTATIC POTENTIAL MAP IN OpenCL

We now present an OpenCL case study based on the DCS kernel in Figure 9.9. This case study is designed to give a CUDA programmer a practical, top to bottom experience with OpenCL. The first step in porting the kernel to OpenCL is to design the organization of the NDRange, which is illustrated in Figure 11.9. The design is a straightforward mapping of CUDA threads to OpenCL work items and CUDA blocks to OpenCL work groups. As shown in Figure 11.9, each work item will calculate up to eight grid points, and each work group will have 64–256 work items. The efficiency considerations in Chapter 9 apply.

The work groups are assigned to the computing units in the same way that CUDA blocks are assigned to the SMs. Such assignment is illustrated in Figure 11.10. One can use the same methodology demonstrated in Chapters 6 through 9 to derive a high-performance OpenCL DCS kernel. Although the syntax is different, the underlying thought process involved in developing a high-performance OpenCL kernel is very much the same as CUDA.

The OpenCL kernel function implementation closely matches the CUDA implementation. Figure 11.11 shows the key differences. One is the `__kernel` keyword in OpenCL versus the `__global` keyword in CUDA. The main difference lies in the way the data access indices are calculated.

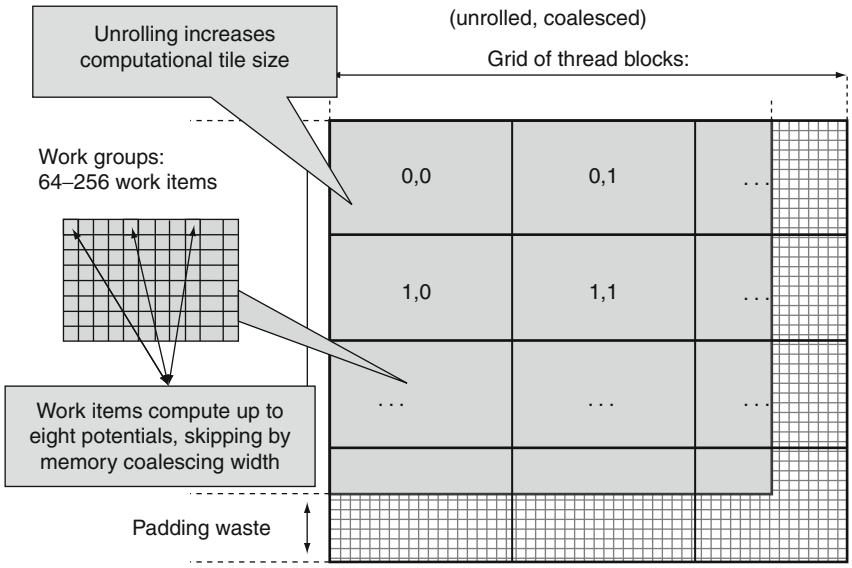

FIGURE 11.9

DCS Kernel Version 3 NDRange configuration.

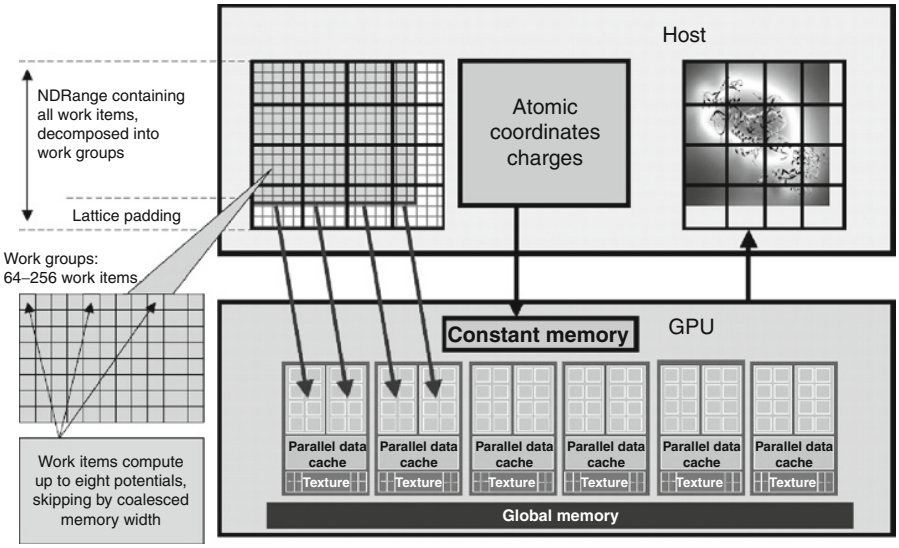

FIGURE 11.10

Mapping DCS NDRange to OpenCL device.

```
OpenCL:
__kernel voidclenergy( …) {

unsigned int xindex= (get_global_id(0) / get_local_id(0))* UNROLLX +
 get_local_id(0) ;
unsigned int yindex= get_global_id(1);
unsigned int outaddr= get_global_size(0) * UNROLLX *yindex+xindex;

CUDA:
__global__ void cuenergy(…) {
Unsigned int xindex= blockIdx.x *blockDim.x +threadIdx.x;
unsigned int yindex= blockIdx.y *blockDim.y +threadIdx.y;
unsigned int outaddr= gridDim.x *blockDim.x * UNROLLX*yindex+xindex
```

FIGURE 11.11

Data access indexing in OpenCL and CUDA.

In this case, the OpenCL `get_global_id(0)` function returns the equivalent of CUDA `blockIdx.x*blockDim.x + threadIdx.x`.

Figure 11.12 shows the inner loop of the OpenCL kernel. The reader should compare this inner loop with the CUDA code in Figure 9.9. The only difference is that the `__rsqrt()` call has been changed to a `native_rsqrt()` call, the OpenCL way to call an intrinsic function on a particular device.

OpenCL adopts a dynamic compilation model. Unlike CUDA, the host program needs to explicitly compile and create a kernel program. This is illustrated in Figure 11.13 for the DCS kernel. Line 1 declares the entire OpenCL DCS kernel source code as a string. Line 3 delivers the source

```
...for (atomid=0; atomid<numatoms; atomid++) {
float dy = coory -atominfo[atomid].y;
float dyz2= (dy * dy) + atominfo[atomid].z;
float dx1 = coorx -atominfo[atomid].x;
float dx2 = dx1 + gridspacing_coalesce;
float dx3 = dx2 + gridspacing_coalesce;
float dx4 = dx3 + gridspacing_coalesce;
float charge = atominfo[atomid].w;
energyvalx1 += charge* native_rsqrt(dx1*dx1 + dyz2);
energyvalx2 += charge* native_rsqrt(dx2*dx2 + dyz2);
energyvalx3 += charge* native_rsqrt(dx3*dx3 + dyz2);
energyvalx4 += charge* native_rsqrt(dx4*dx4 + dyz2);
}
```

FIGURE 11.12

Inner loop of the OpenCL DCS kernel.

OpenCL kernel source code as a big string

```
1 const char* clenergysrc =
    "__kernel__attribute__((reqd_work_group_size_hint(BLOCKSIZEX, BLOCKSIZEY, 1))) \n"
    "void clenergy(__constant int numatoms, __constant float gridspacing, __global float *energy, __constant float4
      *atominfo) { \n"   [...etc and so forth...]
2 cl__program clpgm;
3 clpgm = clCreateProgram WithSource(clctx, 1, &clenergysrc, NULL, &clerr);
  char clcompileflags[4096];
4 sprintf(clcompileflags, "-DUNROLLX=%d -cl-fast-relaxed-math -cl-single-precision-
      constant -cl-denorms-are-zero -cl-mad-enable", UNROLLX);
5 clerr =  clBuildProgram(clpgm, 0, NULL, clcompileflags, NULL, NULL);
6 cl_kernel clkern = clCreateKernel(clpgm, "clenergy", &clerr);
```

Gives raw source code string(s) to OpenCL

Set compiler flags, compile source, and retrieve a handle to the "clenergy" kernel

FIGURE 11.13

Building an OpenCL kernel.

code string to the OpenCL runtime system by calling the `clCreateProgramWithSource()` function. Line 4 sets up the compiler flags for the runtime compilation process. Line 5 invokes the runtime compiler to build the program. Line 6 requests that the OpenCL runtime create the kernel and its data structures so it can be properly launched. After Line 6, `clkern` points to the kernel that can be submitted to a command queue for execution.

As we explained in Section 11.5, OpenCL kernel has a much more complex device management model. We also discussed the creation of context and command queues in the host code. Figure 11.8 actually shows a simple host code that creates a context and command queue for a system with just one GPU to execute the kernel.

Figure 11.14 shows the host code that actually launches the DCS kernel. Line 1 and Line 2 allocate memory for the energy grid data and the atom information. The `clCreateBuffer` function corresponds to the `cudaMalloc()` function. The constant memory is implicitly requested by setting the mode of access to ready only for the `atominfo` array. Note that each memory buffer is associated with a context, which is specified by the first argument to the `clCreateBuffer` function call.

```
1. doutput= clCreateBuffer(clctx, CL_MEM_READ_WRITE,volmemsz,
        NULL, NULL);
2. datominfo= clCreateBuffer(clctx, CL_MEM_READ_ONLY,
        MAXATOMS *sizeof(cl_float4), NULL, NULL);
...
3. clerr= clSetKernelArg(clkern, 0,sizeof(int), &runatoms);
4. clerr= clSetKernelArg(clkern, 1,sizeof(float), &zplane);
5. clerr= clSetKernelArg(clkern, 2,sizeof(cl_mem), &doutput);
6. clerr= clSetKernelArg(clkern, 3,sizeof(cl_mem), &datominfo);
7. cl_event event;
8. clerr= clEnqueueNDRangeKernel(clcmdq,clkern, 2, NULL,
         Gsz,Bsz, 0, NULL, &event);
9. clerr= clWaitForEvents(1, &event);
10. clerr= clReleaseEvent(event);
...
11. clEnqueueReadBuffer(clcmdq,doutput, CL_TRUE, 0,
        volmemsz, energy, 0, NULL, NULL);
12. clReleaseMemObject(doutput);
13. clReleaseMemObject(datominfo);
```

FIGURE 11.14

OpenCL host code for kernel launch and .

Lines 3 through 6 in Figure 11.14 set up the arguments to be passed into the kernel function. In CUDA, the kernel functions are launched with C function call syntax extended with <<<>>>. In OpenCL, there is no explicit call to kernel functions, so one needs to use the `clSetKernelArg()` functions to set up the arguments for the kernel function.

Line 8 in Figure 11.14 submits the DCS kernel for launch. The arguments to the `clEnqueueNDRangeKernel()` function specify the command queue for the device that will execute the kernel, a pointer to the kernel, and the global and local sizes of the NDRange. Lines 9 and 10 check for errors if any. Line 11 transfers the contents of the output data back into the energy array in the host memory. The OpenCL `clEnqueueReadBuffer()` copies data from the device memory to the host memory and corresponds to the device to host direction of the `cudaMemcpy()` function.

The `clReleaseMemObject()` function is a little more sophisticated than `cudaFree()`. OpenCL maintains a reference count for data objects. OpenCL host program modules can retain (`clRetainMemObject()`) and release (`clReleaseMemObject()`) data objects. Note that `clCreateBuffer()` also serves as a retain call. With each retain call, the reference count of the object is incremented. With each release call, the reference count is decremented. When the reference count for an object reaches 0, the object is freed. This way, a library module can hang onto a memory object even though the other parts of the application no longer need the object and thus have released the object.

11.7 SUMMARY

OpenCL is a standardized, cross-platform API designed to support portable parallel application development on heterogeneous computing systems. Like CUDA, OpenCL addresses complex memory hierarchies and data-parallel execution. It draws heavily on the CUDA driver API experience. This is the reason why a CUDA programmer finds these aspects of OpenCL familiar. We have seen this through the mappings of the OpenCL data parallelism model concepts, NDRange API calls, and memory types to their CUDA equivalents.

On the other hand, OpenCL has a more complex platform and device management model that reflects its support for multiplatform and multivendor portability. Although the OpenCL standard is designed to support code portability across devices produced by different vendors, such portability does not come for free. OpenCL programs must be prepared to deal with much greater hardware diversity and thus will exhibit more complexity. We see that the OpenCL device management model, the OpenCL kernel compilation model, and the OpenCL kernel launch are much more complex than their CUDA counterparts.

We have by no means covered all the programming features of OpenCL. The reader is encouraged to read the OpenCL specification [Khronos 2008] and tutorials [Khronos 2010] for more OpenCL features. In particular, we recommend that the reader pay special attention to the device query, object query, and task parallelism model.

We would like to also point the reader to another important parallel programming API for heterogeneous computing systems. Microsoft's DirectCompute is an API that takes advantage of the massively parallel processing power of a modern GPU to accelerate PC application performance in Microsoft® Windows Vista® or Windows 7 [Wikipedia]. DirectCompute is part of Microsoft's DirectX™ collection of APIs. Other DirectX APIs include Direct3D®, Direct2D, DirectWrite, DirectDraw®, DirectMusic®, DirectPlay®, and DirectSound®. DirectCompute is released with the DirectX 11 API but runs on both DirectX 10 and DirectX 11 GPUs. DirectCompute defines several versions of compute shader capabilities to ensure that applications can take advantage of GPUs. The reader should also expect that DirectCompute will have level of complexity similar to that of OpenCL in terms of device management and kernel launch models.

11.8 EXERCISES

11.1 Using the code base in Appendix A and examples in Chapters 3, 4, 5, and 6 to develop an OpenCL version of the matrix–matrix multiplication application.

11.2 Read the "OpenCL Platform Layer" section of the OpenCL specification. Compare the platform querying API functions with what you have learned in CUDA.

11.3 Read the "Memory Objects" section of the OpenCL specification. Compare the object creation and access API functions with what you have learned in CUDA.

11.4 Read the "Kernel Objects" section of the OpenCL specification. Compare the kernel creation and launching API functions with what you have learned in CUDA.

11.5 Read the "OpenCL Programming Language" section of the OpenCL specification. Compare the keywords and types with what you have learned in CUDA.

References and Further Reading

AMD. (2009). *Introductory tutorial to OpenCL™*. Sunnyvale, CA: Advanced Micro Devices. http://developer.amd.com/gpu/ATIStreamSDK/pages/TutorialOpenCL.aspx

Khronos OpenCL Working Group. (2008). Munshi, A. (Ed.), (2008). *The OpenCL specification*, Version 1.0, Document Revision 29. Beaverton, OR: Khronos Group. http://www.khronos.org/registry/cl/specs/opencl-1.0.29.pdf

Khronos Group. (2010). *OpenCL implementations, tutorials, and sample code*. Beaverton, OR: Khronos Group. http://www.khronos.org/developers/resources/opencl/

NVIDIA. (2010). *OpenCL GPU computing support on NVIDIA's CUDA architecture GPUs*. Santa Clara, CA: NVIDIA. http://www.nvidia.com/object/cuda_opencl.html

Wikipedia. (n.d.) *DirectCompute*. http://en.wikipedia.org/wiki/DirectCompute